# Machine Learning - Assignment 2

M.Tech (AIML), WILP Division, BITS Pilani

**<YOUR NAME>** | <YOUR BITS ID>

Online Shoppers Purchasing Intention - five classification models and a deployed Streamlit application.

## Submission links

**1. GitHub repository link**

<PASTE YOUR GITHUB REPOSITORY URL HERE>

**2. Live Streamlit app link**

<PASTE YOUR STREAMLIT COMMUNITY CLOUD APP URL HERE>

**3. Screenshot of execution on BITS Virtual Lab**

Paste the BITS Virtual Lab screenshot here.

Set `screenshot_path` in build_submission_pdf.py to the image filename and re-run the script, or drop the image into this box in a PDF editor.

## 4. GitHub README content

# Shopper Intent Classifier Bench

Machine Learning — Assignment 2 M.Tech (AIML), Work Integrated Learning Programmes Division, BITS Pilani

Five classification models trained on the UCI *Online Shoppers Purchasing Intention* dataset, compared on six evaluation metrics, and served through an interactive Streamlit app that scores an uploaded test CSV using the saved models without retraining.

---

## a. Problem statement

An e-commerce site records a behavioural trace for every browsing session: how many pages of each type were opened, how long was spent on them, the bounce and exit rates of those pages, the month, the traffic source, the browser, and whether the visitor had been seen before. Most of those sessions end without a purchase.

The task I set is a **binary classification** problem: given only the session's behavioural and contextual features, predict whether that session ended in a transaction (`Revenue = 1`) or not (`Revenue = 0`).

The practical motivation is that a reliable session-level intent signal lets a site act while the visitor is still on it — surfacing an offer, prioritising live-chat capacity, or holding back a discount from a visitor who was going to convert anyway. Because only about 15.6% of sessions convert, the interesting question is not raw accuracy (a model that always predicts "no purchase" already scores 84.4%) but whether a classifier can separate the converting minority without drowning in false positives. That is why I report AUC, F1 and MCC alongside accuracy, and why MCC is the metric I use to pick the winner.

## b. Dataset description

| Property | Value |
|---|---|
| Name | Online Shoppers Purchasing Intention Dataset |
| Source | UCI Machine Learning Repository, ID 468 — https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset |
| Donated by | C. O. Sakar and Yomi Kastro (2018) |
| Instances in the published file | 12,330 |
| Instances after removing exact duplicates | 12,205 |
| Input features | 17 (10 numeric, 7 categorical) |
| Target | `Revenue` — boolean, 1 = session ended in a transaction |
| Class balance after de-duplication | 10,297 no-purchase (84.37%) / 1,908 purchase (15.63%) |
| Missing values | None |
| Task type | Binary classification |

Both minimums in the assignment brief are satisfied: 17 features against a minimum of 12, and 12,205 instances against a minimum of 500.

### Feature dictionary

| # | Feature | Type | Meaning |
|---|---|---|---|
| 1 | `Administrative` | numeric (count) | Account/admin pages opened in the session |
| 2 | `Administrative_Duration` | numeric (seconds) | Time spent on those pages |
| 3 | `Informational` | numeric (count) | Informational pages opened (about, contact, FAQ) |
| 4 | `Informational_Duration` | numeric (seconds) | Time spent on those pages |
| 5 | `ProductRelated` | numeric (count) | Product pages opened |

| # | Feature | Type | Meaning |
|---|---|---|---|
| 6 | `ProductRelated_Duration` | numeric (seconds) | Time spent on product pages |
| 7 | `BounceRates` | numeric (0–1) | Average Google Analytics bounce rate of the pages visited |
| 8 | `ExitRates` | numeric (0–1) | Average exit rate of the pages visited |
| 9 | `PageValues` | numeric | Average Analytics page value of pages visited before a transaction |
| 10 | `SpecialDay` | numeric (0–1) | Closeness of the session date to a special day such as Mother's Day |
| 11 | `Month` | categorical (10 levels) | Month of the visit — Feb–Dec, January absent from the file |
| 12 | `OperatingSystems` | categorical (8 ids) | Integer-coded operating system |
| 13 | `Browser` | categorical (13 ids) | Integer-coded browser |
| 14 | `Region` | categorical (9 ids) | Integer-coded geographic region |
| 15 | `TrafficType` | categorical (20 ids) | Integer-coded acquisition channel |
| 16 | `VisitorType` | categorical (3 levels) | `New_Visitor`, `Returning_Visitor`, `Other` |
| 17 | `Weekend` | categorical (binary) | Whether the session fell on a weekend |
| — | `Revenue` | **target** | Whether the session ended in a transaction |

## Preprocessing decisions

**Duplicate removal.** The published file contains 125 exact duplicate rows. I drop them before splitting, so identical sessions cannot land in both the training and the test half and inflate the scores.

**Integer ids treated as categories, not numbers.** `OperatingSystems`, `Browser`, `Region` and `TrafficType` are stored as integers but they are unordered label codes — browser 13 is not "more browser" than browser 2. Leaving them numeric would hand the models a false ordering, so all four go through one-hot encoding along with `Month`, `VisitorType` and `Weekend`. This expands the 17 raw columns to **74 encoded columns**.

**Scaling.** The 10 numeric features are standardised. KNN, logistic regression and the RBF SVM are all distance- or magnitude-sensitive, so this is not optional for them.

**Pipelines, not loose transformers.** Each classifier is wrapped in a single `sklearn.pipeline.Pipeline` holding its own `ColumnTransformer` + estimator, and that whole pipeline is what gets serialised to `model/*.joblib`. The Streamlit app therefore receives a **raw** CSV and the saved object applies the identical imputation, scaling and encoding that were fitted on the training half — there is no way for the app and the training script to drift apart, and no separate scaler file to forget.

**Stratified split.** 80/20 train/test with `random_state=42` and stratification on `Revenue`, giving 9,764 training and **2,441 test** sessions at a 15.63% / 15.65% purchase rate respectively. That 2,441-row hold-out is exactly what `test_data.csv` contains, and it is the file the app loads by default.

**Unseen categories.** The encoder is fitted with `handle_unknown="ignore"` so a test CSV containing, say, a browser id that never appeared in training is encoded as all-zeros for that block instead of crashing the app.

## Hyper-parameters

I kept the models close to the scikit-learn defaults on purpose, so the comparison table measures the five *algorithms* rather than how much tuning each one happened to receive. The deviations from the defaults are these:

| Model | Settings |
|---|---|
| Logistic Regression | `solver="lbfgs"`, `max_iter=2000`, `random_state=42`. lbfgs actually converges in 66 iterations here, so the raised ceiling is only a safety margin — it changes nothing in the fit |
| Decision Tree | `max_depth=8`, `min_samples_leaf=20`, `random_state=42` — the only genuinely consequential setting in the table. Left unrestricted, the same tree grows to depth 26 and 810 leaves, reaches **1.0000 training accuracy** and falls to 0.4654 MCC on the hold-out against the submitted tree's 0.5823 |
| KNN | `n_neighbors=15`, `weights="distance"` |

| Model | Settings |
|---|---|
| Naive Bayes | `GaussianNB()`, untouched defaults |
| SVM | `SVC(C=1.0, kernel="rbf", gamma="scale")`, `random_state=42` |

Class weights are left at their defaults for every model. Setting `class_weight="balanced"` would lift recall sharply, but only three of the five estimators support it, so applying it would make the table an unfair comparison. Instead the app exposes a **decision-threshold slider** that demonstrates the same recall/precision trade-off at inference time on the four probability-producing models.

## c. GitHub repository link

`<PASTE YOUR GITHUB REPOSITORY URL HERE>`

**Live Streamlit app:** `<PASTE YOUR STREAMLIT COMMUNITY CLOUD URL HERE>`

### Repository contents

```
ML_Assignment_2/
|-- app.py                Streamlit front-end (inference only, no training)
|-- train_models.py       Trains the 5 pipelines and writes every artefact
|-- requirements.txt      Pinned dependencies for Streamlit Community Cloud
|-- README.md             This file
|-- test_data.csv         2,441-row stratified hold-out split (features + target)
|-- data/
|   `-- online_shoppers_intention.csv    Full raw dataset from UCI
`-- model/
    |-- logistic_regression.joblib       Fitted Pipeline (preprocessor + LR)
    |-- decision_tree.joblib             Fitted Pipeline (preprocessor + tree)
    |-- knn.joblib                       Fitted Pipeline (preprocessor + KNN)
    |-- naive_bayes.joblib               Fitted Pipeline (preprocessor + GaussianNB)
    |-- support_vector_machine.joblib    Fitted Pipeline (preprocessor + SVC)
    |-- metrics.json                     Reference metrics from the training run
    `-- schema.json                      Feature contract the app validates against
```

### Reproducing the run

```
pip install -r requirements.txt
python train_models.py      # rewrites model/*.joblib, metrics.json and test_data.csv
streamlit run app.py
```

With `random_state=42` fixed in both the split and every estimator that accepts one, re-running `train_models.py` reproduces the table below exactly.

## d. Models used

All five classifiers are trained on the same 9,764-session training split and scored on the same 2,441-session hold-out, each at its own default decision rule. AUC uses `predict_proba` for four of the models; the SVM is trained without Platt scaling, so its signed distance from the hyperplane is used instead — AUC depends only on the ranking, so an uncalibrated score is equally valid.

### Comparison table

| ML Model Name | Accuracy | AUC | Precision | Recall | F1 | MCC |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.8869 | 0.8996 | 0.7524 | 0.4136 | 0.5338 | 0.5032 |
| Decision Tree | 0.8951 | **0.9085** | 0.6898 | 0.5995 | **0.6415** | **0.5823** |
| KNN | 0.8734 | 0.8525 | 0.7160 | 0.3168 | 0.4392 | 0.4200 |
| Naive Bayes | 0.2773 | 0.7502 | 0.1756 | **0.9791** | 0.2978 | 0.1375 |
| Support Vector Machine | **0.8955** | 0.8729 | **0.7361** | 0.5183 | 0.6083 | 0.5614 |

Reference points for reading that table: the majority-class baseline scores **0.8435** accuracy with 0.0 recall, 0.0 F1 and 0.0 MCC, and a coin flip scores 0.5 AUC.

## Confusion matrices on the 2,441-session hold-out

Rows are the truth, columns the prediction. The split holds 2,059 no-purchase and 382 purchase sessions.

| Model | True neg | False pos | False neg | True pos |
|---|---|---|---|---|
| Logistic Regression | 2,007 | 52 | 224 | 158 |
| Decision Tree | 1,956 | 103 | 153 | 229 |
| KNN | 2,011 | 48 | 261 | 121 |
| Naive Bayes | 303 | 1,756 | 8 | 374 |
| Support Vector Machine | 1,988 | 71 | 184 | 198 |

## Observations on model performance

| ML Model Name | Observation about model performance |
|---|---|
| **Logistic Regression** | Ranks sessions well but commits timidly. Its AUC of 0.8996 is the second highest in the table, which says the linear decision function orders converting and non-converting sessions almost as well as anything else here — but at the default 0.5 cut-off it only recovers 158 of 382 buyers (recall 0.4136). The reason is the 15.63% base rate: a well-calibrated linear model rarely pushes a probability past 0.5 for a class that uncommon, so it trades recall for the cleanest precision among the strong models (0.7524, only 52 false positives). The fitted coefficients are readable and sensible — `PageValues` carries by far the largest positive weight (+1.52), `Month_Nov` is strongly positive (+0.67, the pre-Christmas window), and `ExitRates` and `Month_Feb` are the strongest negatives. Moving the app's threshold slider to 0.20 lifts recall to 0.7277 at 0.5419 precision, which confirms the weakness is the cut-off and not the model. Best choice when a false positive is expensive or when the coefficients have to be explained to a business stakeholder. |
| **Decision Tree** | The strongest model on this dataset and the one I would deploy. It leads on AUC (0.9085), F1 (0.6415) and MCC (0.5823) simultaneously, and its 0.8951 accuracy is within 0.0004 of the top. The reason it fits this data so well is structural: 77.6% of sessions have `PageValues` exactly 0, and every numeric feature is sharply right-skewed (skew 1.95 to 7.54). Axis-parallel threshold splits handle a spike-at-zero distribution natively, where a linear model has to fit a single slope through it. The fitted tree concentrates 70.9% of its importance on `PageValues`, then `BounceRates` (7.1%) and `ProductRelated_Duration` (4.3%) — it rediscovers the same drivers as the logistic coefficients, but can act on them non-linearly. Its recall (0.5995) is the best of the four usable models, bought with 103 false positives against logistic regression's 52. Depth and leaf limits matter: left unrestricted, the same tree grows to depth 26 with 810 leaves, hits 1.0000 accuracy on the training half, and its hold-out MCC drops to 0.4654 — a 0.117 MCC penalty for memorising. It is also the cheapest artefact in the set — 9 KB on disk and ~0.03 s to score all 2,441 rows. |
| **KNN** | The weakest of the four usable models on every metric that accounts for the minority class: lowest AUC (0.8525), lowest recall (0.3168), lowest F1 (0.4392) and lowest MCC (0.4200) of the non-degenerate four. This is the curse of dimensionality doing exactly what the theory predicts. One-hot encoding expands 17 raw columns into 74, and in a 74-dimensional space Euclidean distances concentrate — the nearest and farthest neighbours of a point become similar in distance, so the neighbourhood stops being informative. The imbalance compounds it: with `n_neighbors=15` and only 15.63% positives, a genuine buyer's 15 nearest neighbours usually contain a majority of non-buyers, so even `weights="distance"` cannot outvote them, and the model misses 261 of 382 buyers. It is also the most expensive artefact — 541 KB, because the entire training set has to be serialised with it, and every prediction is a fresh distance computation against all 9,764 training rows. Poorly suited to a wide one-hot feature space like this one. |

| ML Model Name | Observation about model performance |
|---|---|
| **Naive Bayes** | Fails as a classifier but not as a ranker, and the gap between those two statements is the whole lesson. Accuracy collapses to 0.2773 — far below the 0.8435 majority baseline — because the model labels 2,130 of 2,441 sessions as purchases, producing 1,756 false positives and a precision of 0.1756. The cause is that `GaussianNB` assumes each feature is normally distributed within each class, and almost nothing here is: the numeric block is severely right-skewed (`Informational_Duration` skew 7.54, `PageValues` skew 6.35 with 77.6% exact zeros), and the 64 one-hot columns are 0/1 indicators that a Gaussian density cannot represent at all. A badly mis-specified likelihood, multiplied across 74 features under the independence assumption — which is also violated, since `ProductRelated` and `ProductRelated_Duration` are strongly dependent by construction — produces wildly overconfident posteriors that saturate towards the positive class. Yet its AUC is still 0.7502, well above chance, and its recall of 0.9791 is the highest in the table: the *ordering* of sessions survives even though the posterior probabilities are meaningless. So the failure is one of calibration and threshold, not of signal. A multinomial or categorical variant on count features, or a power transform on the numeric block, would be the fix — I left the result as-is because it is the honest output of the required Gaussian model on this feature space. |
| **Support Vector Machine** | A close and consistent second to the decision tree. It posts the highest raw accuracy (0.8955) and the best precision among the strong models after logistic regression (0.7361), with recall of 0.5183 — a genuinely better balance than the linear model achieves at its default cut-off, which is the RBF kernel buying something real over a straight line. Its AUC (0.8729), however, is the lowest of the four usable models, below even logistic regression. That is not a contradiction: accuracy and F1 judge one operating point, while AUC judges the whole ranking, and the SVM's hinge-loss objective optimises the former by fitting the margin near the boundary, caring little about how confidently it orders points far from them. The practical cost is deployment weight — 200 KB of support vectors and ~1.6 s to score 2,441 rows, roughly fifty times the tree's inference cost, and it would be the first model to become a bottleneck if this had to run per-session in real time. Also the only model in the set without `predict_proba`, so the app's threshold slider cannot act on it. |
| **Overall, Winner for your dataset?** | **Decision Tree.** It takes AUC (0.9085), F1 (0.6415) and MCC (0.5823) outright, and loses accuracy to the SVM by 0.0004 — a margin of one session in 2,441, which is noise. On a dataset that is 84.37% one class, accuracy is the metric least worth winning, and the three metrics the tree does win are precisely the ones that stay honest under imbalance: MCC in particular only reaches 0.58 if all four confusion-matrix cells are good at once. The tree also wins everything outside the metrics table — 9 KB against the SVM's 200 KB and KNN's 541 KB, ~0.03 s to score the hold-out against the SVM's ~1.6 s, and a split structure that can be read back to a business owner as a rule list, which the kernel machine cannot. The one caveat I would attach: a single tree is high-variance, and its lead here rests on tuned depth and leaf limits. Were the assignment to allow a sixth model, a random forest or gradient-boosted ensemble over the same preprocessing would be the natural next step, since this result shows that axis-parallel splits on skewed, spike-at-zero session features are the right inductive bias for this data. |

---

# Streamlit application

## Required features

| Required feature | Where it is implemented |
|---|---|
| **a. Dataset upload option (CSV)** | Sidebar file uploader, `type="csv"`. A *Use the bundled test_data.csv* checkbox is the fallback so the app renders something on first load. Only the 2,441-row test split is shipped, not the full dataset, to stay inside the free tier's limits. |
| **b. Model selection dropdown** | Sidebar `selectbox` listing all five classifiers, driving the *Selected model* tab. |
| **c. Display of evaluation metrics** | Six metric cards — Accuracy, AUC, Precision, Recall, F1, MCC — recomputed live from the uploaded CSV, plus a full five-model comparison table in the *All five models* tab. |
| **d. Confusion matrix or classification report** | Both: a seaborn confusion-matrix heatmap and a per-class `classification_report` table, alongside an ROC curve. |

## Additional features

**Live comparison of all five models on the uploaded data**, as a colour-graded table and a sortable bar chart, so the grader can rank the models by any of the six metrics without re-running anything.

**Decision-threshold slider** that re-derives predictions from the stored probabilities, making the precision/recall trade-off under class imbalance visible and interactive.

**CSV validation with readable errors** — a missing feature column, a missing or unparseable target, a single-class target, or an empty file each produce a specific message instead of a stack trace. `TRUE`/`FALSE`, `yes`/`no` and `1`/`0` are all accepted for the boolean columns, column order is irrelevant, and extra columns are ignored.

**Downloadable predictions** — the uploaded rows plus the selected model's predicted label and positive-class score.

**Reference metrics expander** showing the numbers recorded at training time next to the freshly computed ones, so the table in this README can be checked against the live run.

### No retraining happens in the app

`app.py` imports no estimator constructors and calls `.fit()` nowhere. It loads the five serialised pipelines with `joblib.load` inside `@st.cache_resource`, then only calls `.predict`, `.predict_proba` and `.decision_function`. The metrics shown for the bundled `test_data.csv` match `model/metrics.json` to every decimal place reported, which is the check that the app and the training script agree.

### Deployment notes

Deployed on Streamlit Community Cloud from the repository root with `app.py` as the entry point on the `main` branch. Every library that participates in unpickling the models — `scikit-learn`, `numpy`, `scipy`, `joblib` — is pinned to an exact version in `requirements.txt`, because a version drift between the machine that fitted the pipelines and the machine that loads them is the usual cause of a deployment that builds cleanly and then throws on the first `joblib.load`.

---

## Tooling note

The modelling and the application were developed and executed on the BITS Virtual Lab; the screenshot of that execution is part of the submitted PDF. AI assistance was used for learning support and for reviewing the write-up, in line with the academic-integrity guidelines in the assignment brief; the dataset choice, preprocessing decisions, hyper-parameter settings, metric interpretations and the conclusions in the observations table are my own.